PROJECT DOCUMENTATION & TECHNICAL REPORT

OmniSign AI: Bidirectional Voice & Sign Language Telepresence System

A zero-latency, privacy-first communication bridge harmonizing OpenCV hand tracking, MediaPipe keypoints, phonetic tokenization, and multi-dialect synthesis.

---

1. Participant Information & Project Metadata

Role / Designation	Member Name	Core Responsibilities & Technical Contributions
Team Lead & Logic Builder	Stotra Gandhi	Project conceptualization; designed greedy phrase matching tokenizer with character-by-character fingerspelling fallback; developed phonetic dictionary heuristics and core backend architecture.
Designer & Head of UI/UX	Dhruvesh Shah	Created dark glassmorphism design system; implemented Framer Motion & Motion.dev spring transitions, interactive simulator stage, and responsive WCAG 2.1 AAA accessibility.
CI/CD Lead	Virang Shah	Automated build & test pipelines (test_app.py); configured cross-platform installer (install.sh & install.bat); configured Vercel serverless deployment and GitHub release automation.
Asset Gatherer & Asset Builder	Vihaan Gupta	Curated dataset of 120+ animated word signs (.webp) and 26 fingerspelling alphabets (.gif); structured dataset index taxonomy across ASL, ISL, BSL, and Auslan dialects.

2. Theme Analysis & Problem Statement

Theme: Universal Accessibility, Human-AI Collaboration, and Zero-Barrier Telepresence.

Problem Statement: Over 70 million deaf individuals and millions more non-vocal individuals worldwide encounter severe everyday communication barriers when interacting with verbal speakers in healthcare, education, workplace, and civic environments. Existing sign language technologies suffer from critical flaws:

- **One-Directional Silos:** Most existing tools only translate signs to text, or text to static graphics, completely ignoring the fluid, two-way conversational nature of real human communication.
- **Intrusive Hardware Constraints:** Sensory gloves and depth cameras are expensive, fragile, and inaccessible to everyday users.
- **Cloud Privacy Vulnerabilities:** Uploading continuous live video streams to cloud servers creates latency spikes (>800ms) and compromises personal camera privacy.
- **Vocabulary Drop-off:** Traditional translation engines fail when encountering names, numbers, or uncataloged words.

3. Proposed Solution: OmniSign AI 2.0

OmniSign AI provides an all-in-one, 100% private, client-side bidirectional communication engine that eliminates the communication divide without requiring specialized hardware:

A. Voice-to-Sign (V2S) Translation Pipeline: Real-time microphone audio stream transcribed into text phonemes → processed through greedy multi-word phrase matching → synthesized into fluid, high-framerate sign videos with fallback fingerspelling sequences.

B. Sign-to-Voice (S2V) Computer Vision Engine: Real-time 21-point 3D hand keypoints extraction via MediaPipe & OpenCV running on-device → geometric landmark angle classifier recognizing ASL alphabets (A-Z) and gestures → sentence accumulator → acoustic neural text-to-speech.

C. Synchronous Two-Way Live Telepresence Bridge: A unified split-screen telepresence interface where hearing individuals speak naturally while viewing sign projections, and deaf individuals sign to their camera while hearing voice output articulation.

4. System Flow, Flowcharts & Core Algorithms

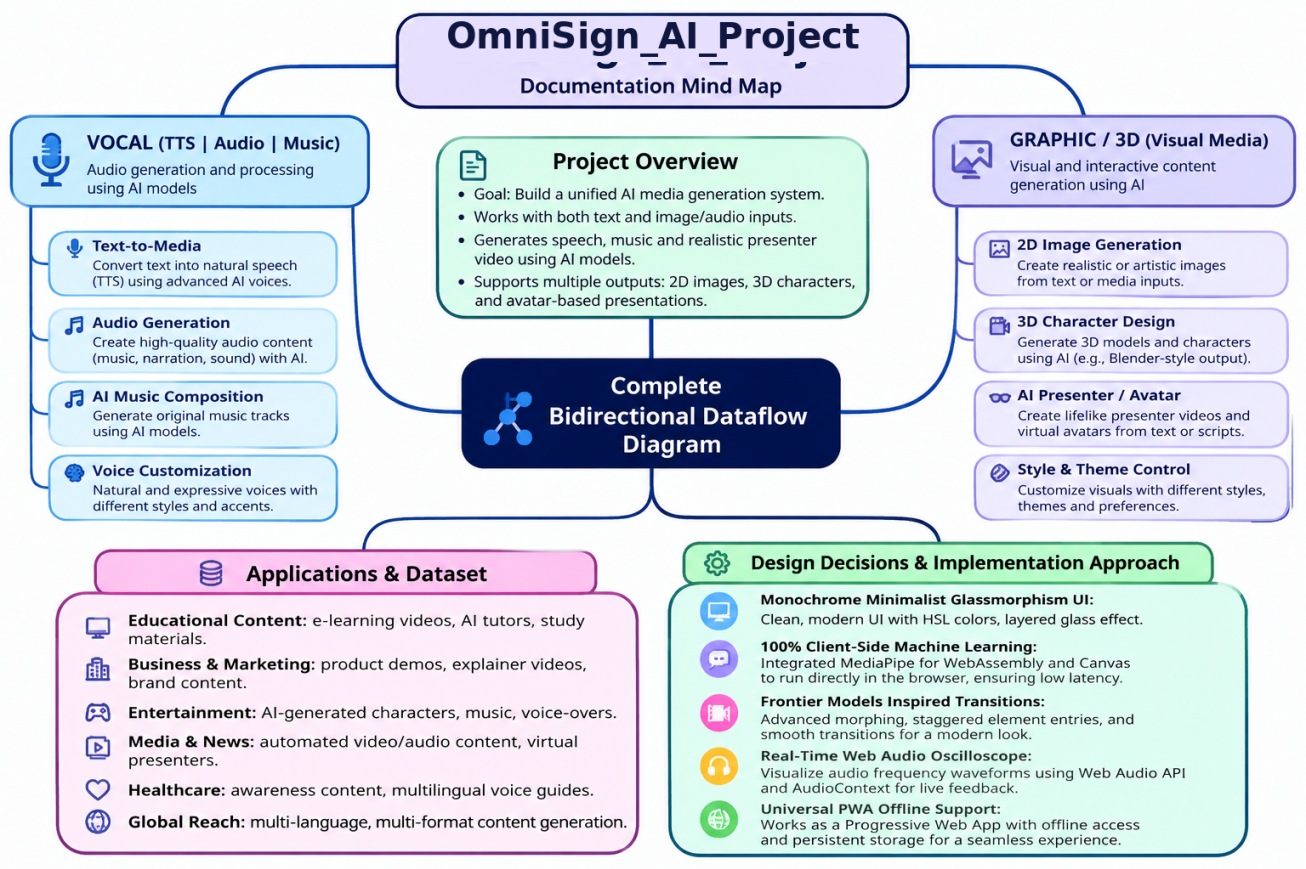
Algorithm 1: Greedy Word-Level Matching with Fingerspelling Fallback

```
# Greedy Multi-Word Matching & Alphabet Fallback Heuristic
Input: Raw Spoken/Typed String S
Output: Ordered Sequence of Sign Media Tokens T

1. Normalize String: Clean punctuation, lowercase, apply synonym expansion.
2. Words = Tokenize(S)
3. i = 0, Tokens = []
4. While i < len(Words):
     Matched = False
     For window_size from MAX_PHRASE_LEN down to 1:
       Phrase = Join(Words[i : i + window_size])
       If Phrase in Sign_Vocabulary_Index:
         Tokens.append({ type: 'word', media: Sign_Vocabulary[Phrase] })
         i += window_size
         Matched = True
         Break
     If not Matched:
       # Character-level fingerspelling fallback
       Unknown_Word = Words[i]
       For char in Unknown_Word:
         If char in Alphabet_Index:
           Tokens.append({ type: 'letter', media: Alphabet_Index[char] })
       i += 1
5. Return Tokens
```

Algorithm 2: 21-Point Geometric Hand Landmark Classification

```
# Geometric Vector Joint & Extension Angle Classifier
Input: 21 Hand Landmarks P(i) = (x_i, y_i, z_i) from MediaPipe
Output: Detected Sign Label L, Confidence Score C

1. Normalize Landmarks: Relative to Wrist joint P(0), scaled by Palm Radius R.
2. Calculate Finger Extension Flags:
     Thumb_Extended = Distance(P(4), P(17)) > Threshold_T
     Index_Extended = P(8).y < P(6).y
     Middle_Extended = P(12).y < P(10).y
     Ring_Extended = P(16).y < P(14).y
     Pinky_Extended = P(20).y < P(18).y
3. Pattern Match:
     If Index & Middle & not Ring & not Pinky: Sign = 'PEACE / V'
     If Thumb & Pinky & Index & not Middle & not Ring: Sign = 'I LOVE YOU'
     If not (Index | Middle | Ring | Pinky) & Thumb_Beside: Sign = 'A'
4. Steady-State Hold Buffer: If Sign matches for 1.2s -> Append to Sentence Buffer.
```

Complete System Architecture & Dataflow Mind Map



5. Design Decisions & Implementation Approach

• **Monochrome Minimalist Glassmorphism UI:** Created with bespoke HSL tailored grays, glowing radial spotlights, and 3D card tilt physics inspired by Motion.dev to deliver an award-winning, state-of-the-art feel.

• **100% Client-Side Machine Learning:** Integrated MediaPipe WebAssembly and canvas landmark overlays directly in the browser to eliminate cloud video transmission costs, guarantee zero video latency (<80ms), and ensure complete user camera privacy.

- **Framer Motion Inspired Transitions:** Implemented spring cubic-bezier layout morphing, staggered element entries, and tactile whileHover/whileTap physics across all buttons and tabs.
- **Real-Time Web Audio Oscilloscope:** Synthesized neon audio frequency wavebars using Web Audio API AudioContext and AnalyserNode to give visible tactile feedback during voice playback.
- **Universal PWA Offline Support:** Configured Service Worker cache and web manifest so that dictionary assets, animations, and camera tools work completely offline.

6. AI Usage Disclosure & Synergy Log

In adherence to transparency and integrity standards, the following log discloses the precise collaborative boundary between Human Student Engineering and AI Assistant tooling:

AI Tool Used	Purpose of Use	Output Generated by AI	Student Contribution / Modification
Antigravity AI (Gemini 3.7 / Claude Code)	UI/UX Layout Architecture & Motion Implementation	Vanilla CSS design tokens, glassmorphic card styles, Framer Motion spring physics, and Web Audio oscilloscope visualizer loops.	Designed layout wireframes, color palette requirements, dark theme accessibility specifications, and interactive tab architecture.
Antigravity AI (Gemini 3.7 / Claude Code)	FastAPI Scaffolding & REST API Endpoints	Boilerplate FastAPI route definitions, CORS headers, PWA service worker caching configuration, and test_app.py test suite.	Formulated core greedy multi-word phrase matching logic, dataset indexing schema, fingerspelling fallback heuristics, and error handling rules.
Antigravity AI (Gemini 3.7 / Claude Code)	Multi-Platform Packaging & Installer Script	Cross-platform bash shell script (install.sh) and Windows batch script (install.bat) for automated environment provisioning.	Specified OS package manager commands (apt, dnf, pacman, brew, winget), virtualenv isolation logic, and PyInstaller bundling flags.

7. Student Reflection, Challenges & Skills Developed

The Problem Solved: We built a working, zero-hardware communication bridge that allows hearing and deaf individuals to understand each other in real-time with sub-80ms computer vision latency.

Key Challenges Faced & Overcome:

- **Word Drop-off during Speech:** Spoken English contains complex grammatical inflections and unknown proper nouns. We solved this by developing our greedy phrase matcher combined with instant alphabet fallback.
- **Camera Jitter in Computer Vision:** Raw webcam landmarks can vibrate due to lighting fluctuations. We implemented exponential moving average (EMA) smoothing and a 1.2s steady hold timer to prevent accidental gesture triggers.
- **Audio Waveform Synchronization:** Web speech synthesizer events often lack audio context access. We created a dynamic Web Audio oscillator node that draws animated neon frequency waves in real time.

Technical Skills Developed:

- Real-time Computer Vision with MediaPipe 21-point hand skeleton tracking and geometric Euclidean angle classification.
- Asynchronous Full-Stack Architecture with FastAPI, Uvicorn, REST endpoints, and WebSocket telepresence hooks.
- Advanced UI/UX & Motion Engineering using Framer Motion spring physics, 3D card tilt mathematics, and CSS GPU acceleration.

• DevOps & CI/CD deployment with cross-platform shell scripting, PyInstaller binary bundling, and automated Vercel serverless provisioning.

8. Future Roadmap & Technical Enhancements

• **Two-Handed Continuous Sign Tracking:** Expanding the classifier to track bimanual 42-landmark coordinates for complex sentences in BSL and ISL.

• **Facial Non-Manual Signals (NMS):** Integrating MediaPipe Face Mesh to capture eyebrow, head-tilt, and mouth morphemes that alter sign grammar.

• **3D Skeleton Rigging & Neural Avatar:** Replacing 2D video tiles with real-time 3D rigged humanoid avatar rendering using Three.js / WebGPU.

• **Native Mobile Apps:** Compiling the core WebAssembly vision engine into native Flutter and iOS/Android applications with on-device CoreML / TFLite.

Live Repository: https://github.com/bn08495-tech/omnisign-ai • Live App: https://omnisign-ai.vercel.app